

KERAS 및 YOLOv5를 이용한 Deep_Learning 객체 인식 리포트

작성자: 서예현

사용한 도구

딥러닝이란?

Keras를 이용한 딥러닝

YOLO를 이용한 딥러닝

1차~4차 시도

프로젝트를 마치며

Reference

AI는 어느 순간부터 사람들의 생활에 깊이 들어오게 되었다. AI를 탑재한 가전제품에 이어 실시간 채팅형 AI(챗GPT, 제미나이 등)가 등장하고, 이제는 사이트나 회사에서도 상담사가 아닌 AI를 먼저 연결시켜주곤 한다. AI와 전화를 하거나 상담을 통해 AI의 작동 구조에 대해 호기심을 갖게 되는 계기가 되었다. 해당 리포트에서는 DW아카데미에서 학습, 실습 한 KERAS와 YOLO를 통한 객체 인식에 대한 딥러닝 결과를 정리, 기록하고자 한다.

사용한 도구

언어: Python

라이브러리: openCV, YOLO

모듈: PyTorch, KERAS

딥러닝이란?

딥러닝은 머신러닝의 한 분야이다. 그러나 도메인 지식을 활용하여 데이터의 특징을 추출하고 디자인하는 전통적인 머신러닝과는 달리, 복잡한 데이터 특성을 스스로 학습, 추출하는 심층 신경망을 사용한다.

딥러닝의 경우, 많은 파라미터와 데이터가 필요하여 상당한 연산량과 시간이 소요되곤 한다. 또한 심층 신경망을 훈련시키기 위한 하이퍼파라미터를 찾기 위한 시행착오가 필요하다. 작동 방식이 복잡하여 예측에 중요한 역할을 하는 요소를 파악하기도 어려워, 신뢰성에 대한 문제로 이어진다. 실생활에 AI 모델을 적용하는 데 법적, 제도적 걸림돌이 생긴 이유이다.

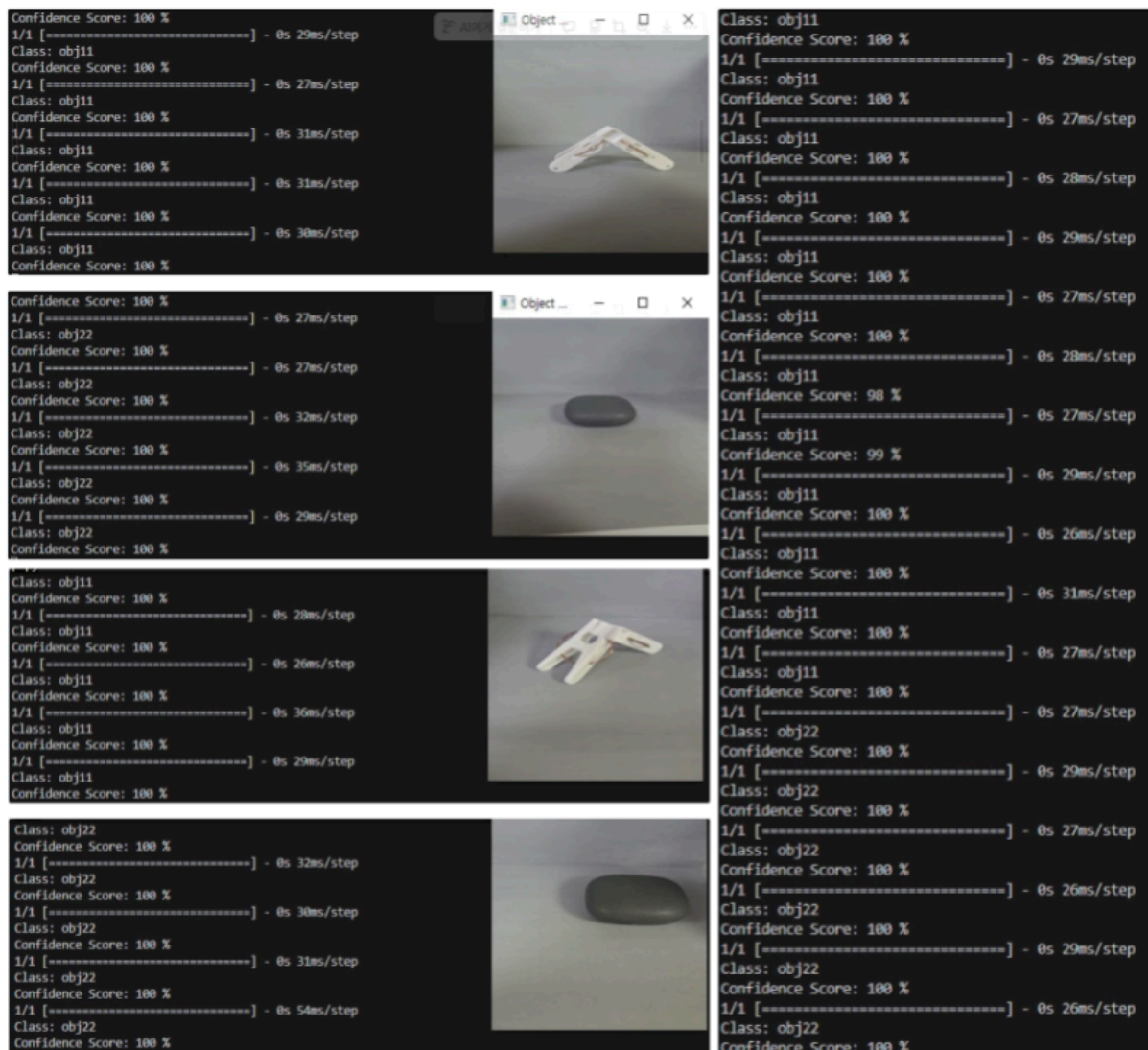
이러한 단점에도 불구하고 딥러닝의 스스로 특징을 추출하는 성능은 기존의 머신러닝에 비해 월등히 우수한 성능을 보인다. 이는 딥러닝이 인공지능 방법론의 대표 주자로 자리매김하게 된 이유이다.

딥러닝 입문

딥러닝을 위한 데이터를 수집하기 위하여 openCV를 이용하여 영상을 불러와 캡처하는 방식을 이용하였다. 각 객체의 형태를 최대한 자세하게 영상에 담고, 해당 영상을 openCV로 불러와 캡처한 화면을 object 파일에 저장하는 방식을 사용하였다.

Keras를 이용한 딥러닝

YOLO를 이용한 딥러닝을 진행하기에 앞서, KERAS를 이용한 딥러닝 연습을 선행하였다. KERAS를 이용한 딥러닝을 진행하기 위해, 두 개의 객체를 준비하였다. 각 객체를 obj_01, obj_02로 지정한 후, 'Teachable Machine'을 이용하여 학습 모델을 작성하였다. 이후 KERAS를 호출해 모델을 학습 시킨 후, 학습 결과 확인을 위해 촬영한 검증 영상을 통해 학습 결과를 검증하였다.



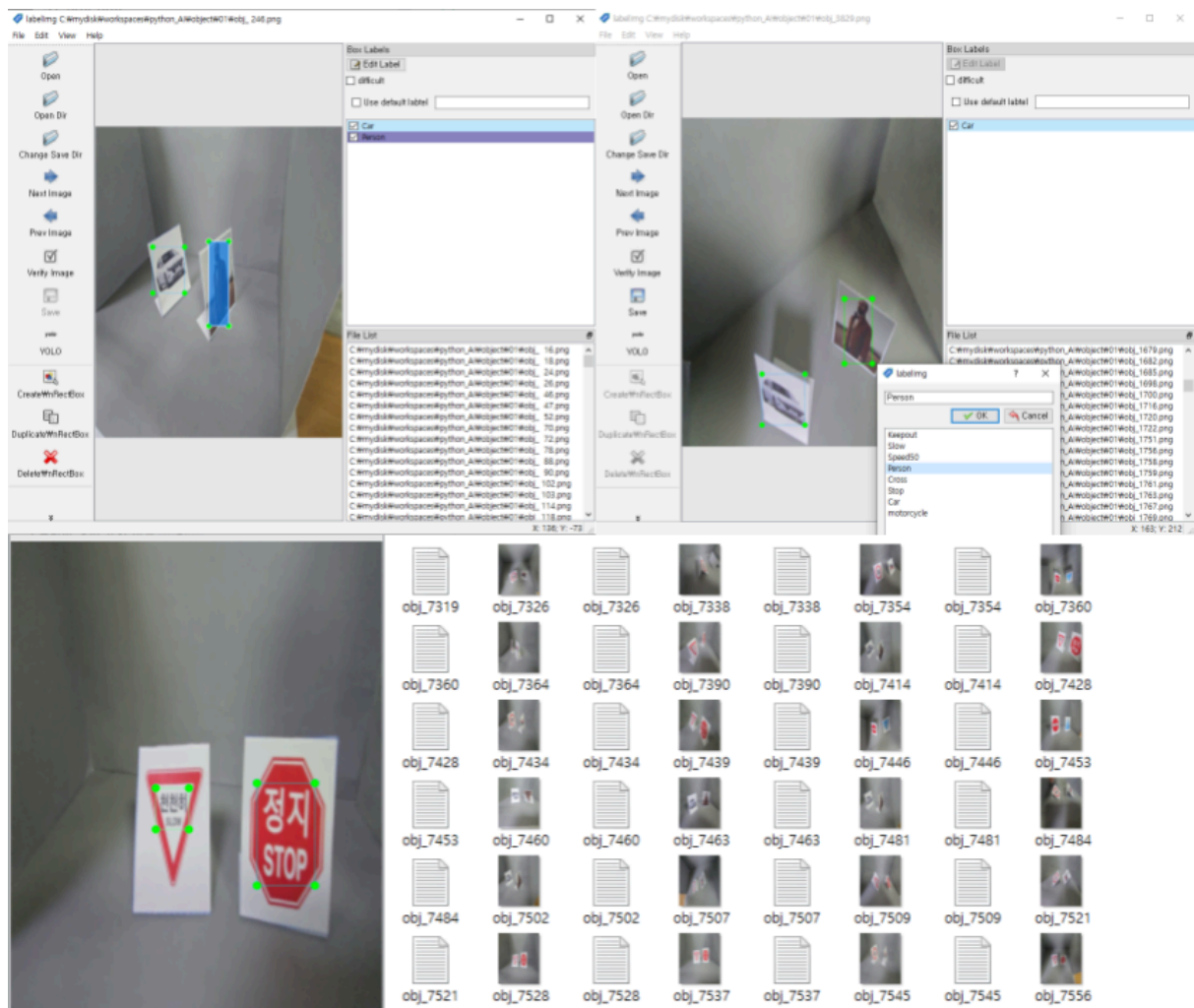
촬영된 영상을 이용한 검증(좌)/ESP32 캠을 이용한 검증(우)

검증 결과, 각 오브젝트를 확실하게 인식하는 것으로 성공적으로 해당 이미지를 학습하였음을 알 수 있었다. 다만 KERAS의 경우 한 번에 하나의 객체만을 분류할 수 있었다. 그렇다면 복수의 객체를 한 번에 분류하기 위해서는 무엇을 사용해야 할까?

YOLO를 이용한 딥러닝

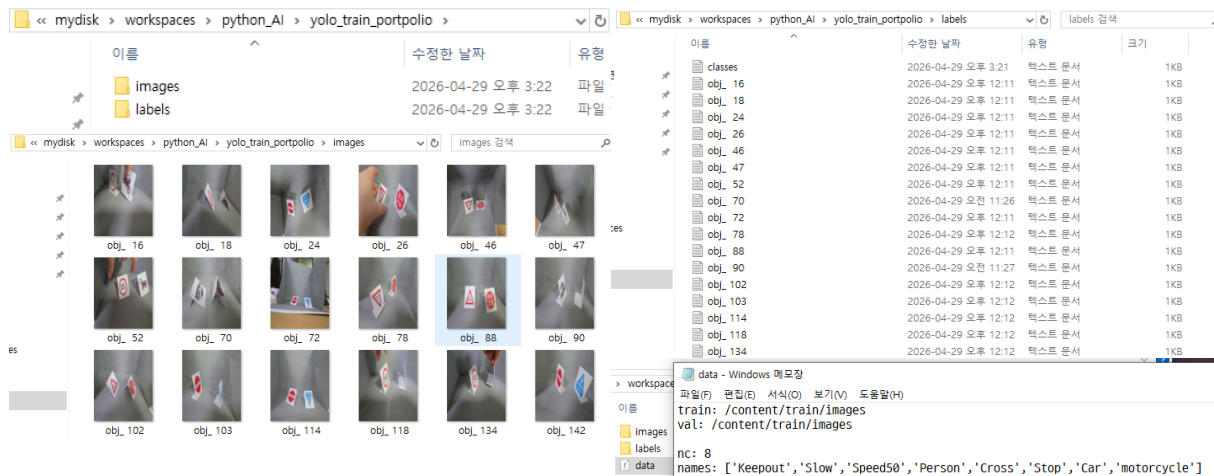
복수의 이미지를 학습시키기 위해서 선택한 모듈은 YOLO였다.

YOLO를 이용하는 경우에는 KERAS와 방식이 다소 변화된다. 우선, 동영상상을 로드하여 객체들의 형태를 캡처하는 것은 동일하다. 이후, labeling 프로그램을 이용하여 분류하고자 하는 이미지들의 라벨링을 진행하게 된다.



labelimg를 이용해 진행한 라벨링

이러한 방식으로 라벨링을 진행한 이미지와 라벨 파일을 각각 train/images, train/labels 파일로 옮겨 놓는다. train 폴더 내부에는 data.yaml 파일을 만들어, 내부에 학습시킬 데이터와 라벨 등의 내용을 채워넣는다.



train 폴더(좌상단)/images 폴더(좌하단)/labels 폴더(우상단)/data.yaml 내용 (우하단)

YOLO의 학습 모델을 생성하는 데에는 GPU를 사용하는 것이 시간적인 문제를 확연히 줄일 수 있다. 다만, GPU가 탑재되지 않은 경우에 대비하기 위하여, 학습 모델의 작성은 Google에서 제공하는 Colab을 이용하게 되었다.

```

[ ] !unzip /content/yolo_train.zip
> 숨겨진 출력 표시

[ ] !git clone https://github.com/ultralytics/yolov5.git
> 숨겨진 출력 표시

[ ] !pip install -U -r yolov5/requirements.txt
> *** 숨겨진 출력 표시

[ ] !pip uninstall torch
> 숨겨진 출력 표시

[ ] !pip uninstall torchvision
> 숨겨진 출력 표시

[ ] !pip install torch==2.10
> 숨겨진 출력 표시

[ ] !pip install torchvision==0.25
> 숨겨진 출력 표시

[ ] !python yolov5/train.py --data "/content/data.yaml" --epochs 30 --batch 16 --cfg "/content/yolov5/models/yolov5s.yaml"
> 숨겨진 출력 표시

```

Colab에서 작성한 코드

학습 모델을 만들기 위해 사진에 나온 코드를 이용하였다. yolo_train.zip 파일은 images 폴더와 labels 폴더가 들어있는 train 폴더와, 학습 모델 작성을 위한 정보가 담겨 있는 data.yaml 파일로 구성되어 있다.

YOLO는 YOLOv5를 이용하였기 때문에 github에서 해당 모듈을 다운로드 받았고, YOLOv5 내부의 requirements.txt를 이용하여 필요한 모듈과 라이브러리를 다운로드 받을 수 있었다.

다만 Torch와 Torchvision의 경우 필요 이상의 버전을 사용하게 되어 오류가 발생하였다. 이를 해결하기 위하여 Torch 및 Torchvision을 삭제 후 필요한 버전으로 재설치를 진행하게 되었다. 모든 모듈 및 라이브러리가 정상 설치된 이후 학습 모델 작성을 실시하니, 원하는 대로 학습이 진행 되는 것을 확인할 수 있었다.

```
--data '/content/data.yaai' --epochs 30 --batch 16 --cfg '/content/yolov5/models/yolov5s.yaai'
...
with torch.cuda.amp.autocast(amp):
  3/29 4.176 0.0512 0.01662 0.01515 48 640: 99% 69/70 [00:23<
with torch.cuda.amp.autocast(amp):
  3/29 4.176 0.0507 0.01662 0.01506 26 640: 100% 70/70 [00:24<
Class Images Instances P R mAP50 mAP50-95: 100% 35,
all 1111 2020 0.455 0.772 0.562 0.223

Epoch GPU_mem box_loss obj_loss cls_loss Instances Size
0x 0/70 [00:00< 711x/s]/content/yolov5/train.py:414: FutureWarning: torch.cuda.amp.autocast(a
with torch.cuda.amp.autocast(amp):
  4/29 4.176 0.0584 0.01565 0.008697 48 640: 1% 1/70 [00:00<
with torch.cuda.amp.autocast(amp):
  4/29 4.176 0.06131 0.01622 0.009162 54 640: 3% 2/70 [00:00<
with torch.cuda.amp.autocast(amp):
  4/29 4.176 0.06036 0.01557 0.009893 44 640: 4% 3/70 [00:00<
with torch.cuda.amp.autocast(amp):
  4/29 4.176 0.05954 0.01552 0.009893 49 640: 6% 4/70 [00:01<
with torch.cuda.amp.autocast(amp):
  4/29 4.176 0.05954 0.01528 0.009813 56 640: 7% 5/70 [00:01<
with torch.cuda.amp.autocast(amp):
  4/29 4.176 0.05971 0.01705 0.009675 66 640: 9% 6/70 [00:01<
with torch.cuda.amp.autocast(amp):
  4/29 4.176 0.06015 0.01662 0.0099 44 640: 10% 7/70 [00:01<
with torch.cuda.amp.autocast(amp):
  4/29 4.176 0.05971 0.01653 0.009955 52 640: 11% 8/70 [00:02<
with torch.cuda.amp.autocast(amp):
  4/29 4.176 0.06189 0.01611 0.01016 40 640: 13% 9/70 [00:02<
with torch.cuda.amp.autocast(amp):
  4/29 4.176 0.06235 0.01581 0.01011 42 640: 14% 10/70 [00:02<
with torch.cuda.amp.autocast(amp):
  4/29 4.176 0.06197 0.01559 0.01004 41 640: 16% 11/70 [00:03<
with torch.cuda.amp.autocast(amp):
  4/29 4.176 0.06176 0.01539 0.01005 38 640: 17% 12/70 [00:03<
with torch.cuda.amp.autocast(amp):
  4/29 4.176 0.06156 0.01548 0.01017 47 640: 19% 13/70 [00:03<
with torch.cuda.amp.autocast(amp):
  4/29 4.176 0.06189 0.01554 0.01017 61 640: 20% 14/70 [00:03<
with torch.cuda.amp.autocast(amp):
  4/29 4.176 0.0612 0.01584 0.01009 66 640: 21% 15/70 [00:04<
with torch.cuda.amp.autocast(amp):
  4/29 4.176 0.05777 0.01581 0.009587 47 640: 23% 16/70 [00:04<
```

Validating yolov5/runs/train/exp2/weights/best.pt...
Fusing layers...
YOLOv5s summary: 157 layers, 7031701 parameters, 0 gradients, 15.8 GFLOPs

Class	Images	Instances	P	R	mAP50	mAP50-95: 100% 35,
all	1111	2020	0.92	0.929	0.925	0.642
Keepout	1111	239	0.866	0.874	0.879	0.606
Slow	1111	310	0.808	0.803	0.809	0.495
Speed50	1111	271	0.859	0.86	0.821	0.516
Person	1111	219	0.959	0.991	0.992	0.77
Cross	1111	216	0.977	0.966	0.973	0.563
Stop	1111	288	0.94	0.955	0.94	0.633
Car	1111	236	0.989	0.996	0.992	0.786
motorcycle	1111	241	0.96	0.988	0.992	0.766

Results saved to yolov5/runs/train/exp2

학습 모델이 만들어지고 있는 모습(좌)과 학습 모델 작성이 완료되었음을 나타내는 메세지(우)

```
import torch
import cv2
import pathlib # 추가
from numpy import random

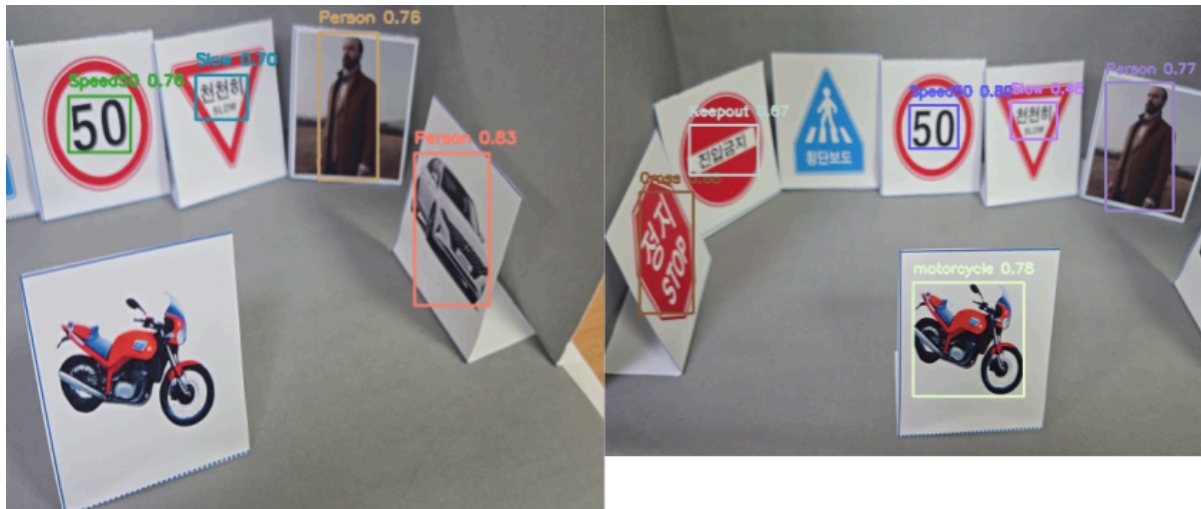
# 1. 경로 호환성 해결
pathlib.PosixPath = pathlib.WindowsPath

# YOLOv5 모델 정의
# model = torch.hub.load('ultralytics/yolov5', 'yolov5s', pretrained=True)
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
model = torch.hub.load('ultralytics/yolov5', 'custom', path='best.pt', force_reload=True)
model.to(device)
```

모델 학습을 위한 코드

작성 된 학습 모델은 위와 같은 코드를 이용하여 활성화 시킬 수 있다. 학습 과정에서는 GPU를 사용하였으나, 모델의 활성화는 CPU에서 진행되므로 cuda를 실행 시킬 수 있는가에 대한 여부를 확인하는 절차가 추가되었다.

1차 시도



해당 코드를 이용하여 학습시킨 모델로 검증용 동영상을 실행시킨 결과는 위와 같다.

정지 표지판을 횡단보도 표지판으로 인식하고, 횡단보도 표지판은 인식을 하지 못하였다. 또한 서행 표지판의 경우에는 학습 신뢰도가 0.48 정도로 매우 낮은 수치를 보였고, 전체적으로 80%의 신뢰도를 넘지 못하였다.

이는 화질 저하에 따른 가독성 저하를 고려하지 않은 채 학습한 것을 원인으로 생각하였다. 문제를 해결하기 위해 화질이 선명할 수 있도록 동영상을 촬영할 때 객체별 촬영 시간과 카메라의 움직임에 신경 썼고, 객체 별로 식별이 용이할 것이라 판단되는 조건으로 라벨링을 진행하였다. 대표적으로 서행 표지판의 경우 화질이 낮아지며 한글이 잘 보이지 않는 경우를 고려해 붉은 역삼각형 자체를 식별하도록 라벨링 하였다. 또한 이미지가 온전하지 않더라도 형태가 40% 이상 보이며, 식별이 가능한 상태일 경우에는 라벨링을 진행하여 각 객체의 형태에 대한 데이터를 넓히고자 하였다.

2차 시도



2차로 진행한 라벨링만을 이용한 결과, 전체적인 신뢰도는 확연히 높아진 것을 볼 수 있다. 그러나 정지 표지판을 횡단보도 표지판으로 인식하거나, 사람을 자동차와 오인하는 등의 문제점은 남아있었다.

1차 학습 모델에 비하여 데이터 양이 부족한 것이 원인이라 생각하여 1차 라벨링과 2차 라벨링을 통합하여 재학습을 진행하였다.

3차 시도



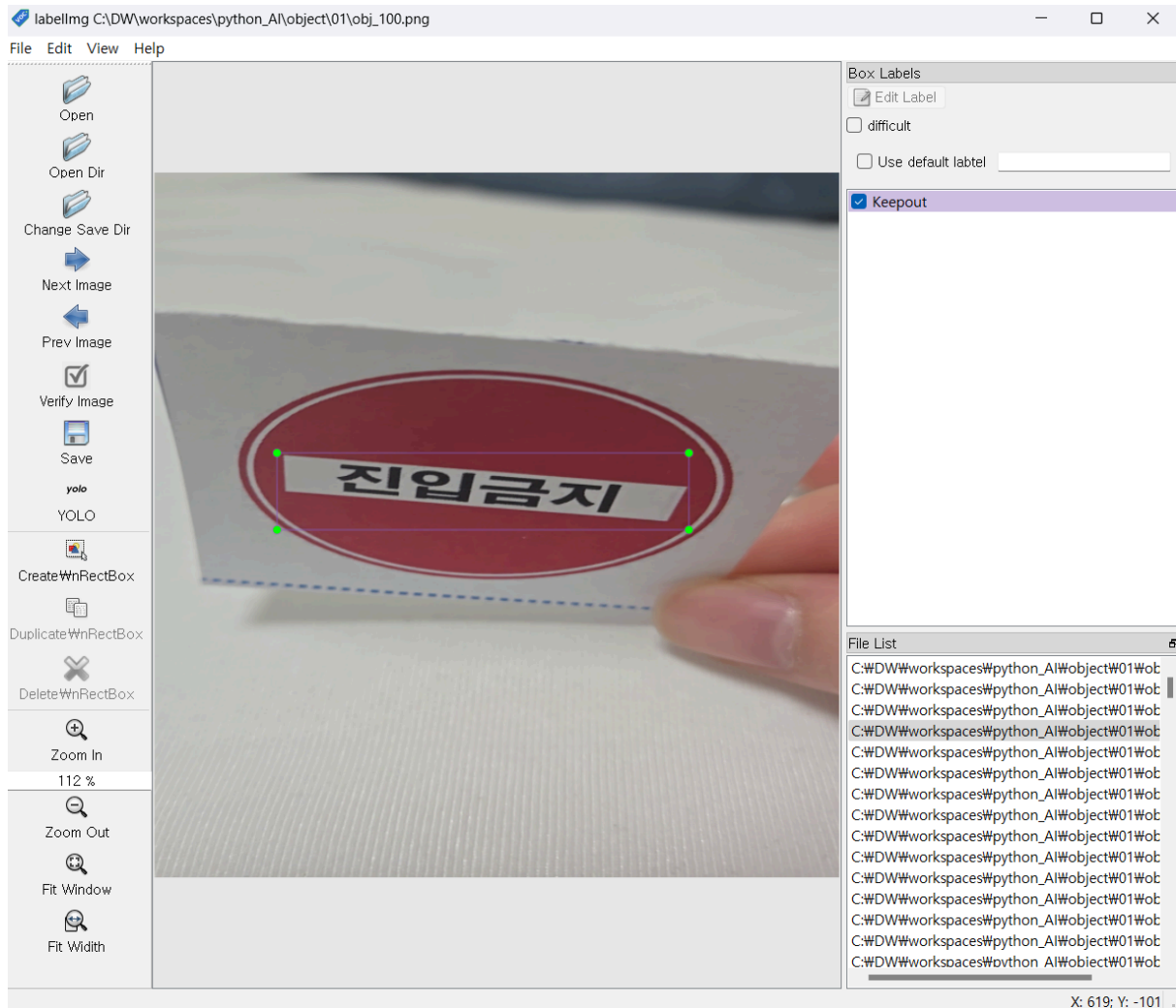
1차 학습 결과와 2차 학습 결과를 통합하여 인식 시킨 결과는 2차 학습 결과만으로 검증한 것 보다 신뢰율이 떨어지는 것을 볼 수 있었다. 정지 표지판과 서행 표지판은 인식하지 못하였고, 해당 검증에서는 자동차를 사람과 혼동하는 결과를 볼 수 있었다.

학습을 위한 라벨링 작업 중, 이미지 자체적인 해상도 저하로 인한 문제일 수도 있다는 생각이 들었다. 해당 동영상은 224, 284의 크기로 캡처를 진행하고 있었으나, 해상도의 조정을 위하여 640, 680으로 조정하여 다시금 라벨링을 진행하였다.

4차 시도

```
frame=cv2.resize(frame,(640,660),interpolation=cv2.INTER_AREA)
```

해상도 상향을 위해 설정 변경



1차, 2차 라벨링보다 확연히 높은 해상도를 확인할 수 있었다. 해당 라벨링 작업의 경우 표지판 내부의 문자 및 기호에 집중하여 라벨링을 진행하였다.



지금껏 진행한 학습 중 가장 높은 수치의 학습 결과를 보여준다. 횡단보도와 정지 표지판이 함께 있는 경우에는 인식을 잘 하지 못하거나, 배경이 없는 경우 일정 거리 이상 멀어졌을 때 아무것도 인식을 하지 못하는 등의 문제점은 여전히 존재했다.

4차 시도를 통하여 단순히 많은 데이터나 단순히 좋은 데이터가 아닌, 충분한 질을 갖춘 많은 데이터가 필요함을 절실히 깨달았다.

프로젝트를 마치며

해당 프로젝트를 통하여 Keras 및 YOLO등의 도구를 이용한 딥러닝의 방법을 몸에 익힐 수 있었다. 다만, 원하는 결과인 모든 객체의 인식률 0.90에는 도달하지 못한 점이 큰 아쉬움으로 남는다. 그러나 데이터의 정제가 얼마나 중요한지 깨달을 수 있었다.

이전까지는 충분히 정제된 데이터를 사용했기에 별다른 전처리 과정을 겪을 일이 없었으나, 이번에는 스스로 데이터를 정제해야만 했다. 그 과정에서 어떤 데이터가 좋은 데이터일지 고민할 수 있었고, 데이터의 질을 끌어올리기 위해 시행착오를 겪기도 했다. 이러한 경험을 토대로 이후에는 조금 더 질이 좋은 데이터를 조금 더 다양하게 수집할 수 있을 것이라 생각한다.

Reference

[All Around AI 4편] 딥러닝의 이해

<https://news.skhynix.co.kr/all-around-ai-4/>